

Reviewer Pack — Harmonic Coefficient Sum Lower Bound for Resolution Proof Le...

Entry 161db2211085 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 17:37:12 UTC

Harmonic Coefficient Sum Lower Bound for Resolution Proof Length

Entry ID: 161db2211085 Verdict: INCONCLUSIVE Recorded: 2026-05-10 17:37:12 UTC

1. Conjecture

Field A (mathematical branch): Harmonic Analysis on Finite Groups **Field B** (complexity object): Resolution Proof Length

Statement:

For any 3-SAT instance with n variables, the resolution proof length is at least the sum of absolute values of its Fourier coefficients over the hypercube group $\{0,1\}^n$. Formally: $\text{length}(\Pi) \geq \sum_{S \subseteq [n]} |\hat{f}(S)|$, where f is the indicator function of satisfying assignments.

Rationale (proposer’s reasoning):

Harmonic analysis reveals the function’s structure via Fourier coefficients, which may correlate with proof complexity. The sum of absolute coefficients quantifies the function’s ‘spread’ over the hypercube, potentially reflecting the number of clauses needed to resolve.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4b560ffa58b8276d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math

def fast_walsh_hadamard_transform(f):
    n = len(f)
    if n == 1:
        return f
    even = fast_walsh_hadamard_transform([f[i] for i in range(0, n, 2)])
    odd = fast_walsh_hadamard_transform([f[i] for i in range(1, n, 2)])
    result = [0] * n
    for k in range(n // 2):
        result[k] = even[k] + odd[k]
        result[k + n // 2] = even[k] - odd[k]
    return result

def fourier_coefficients(f):
    n = len(f)
    F = fast_walsh_hadamard_transform([f[i] / math.sqrt(n) for i in range(n)])
    return F

def indicator_function(x, n):
    return 1 if all(x[i] == 0 for i in range(n)) else 0

def generate_3sat_instance(n):
    clauses = []
    for _ in range(2 * n):
        clause = [random.choice([-1, 1]) * random.randint(1, n) for _ in range(3)]
        clauses.append(clause)
    return clauses

def dpll_solve(clauses, assignment):
    if not clauses:
        return True
    literal = next(lit for lit in range(-n, n + 1) if lit != 0 and lit not in assignment)
    pos_lit = abs(literal)
    if literal > 0:
        assignment[pos_lit] = True
    else:
        assignment[-pos_lit] = False
```

```

new_clauses = [c for c in clauses if not any(abs(lit) == pos_lit for lit in c)]
if dpll_solve(new_clauses, assignment):
    return True
del assignment[pos_lit]
if literal > 0:
    assignment[pos_lit] = False
else:
    assignment[-pos_lit] = True
new_clauses = [c for c in clauses if not any(abs(lit) == pos_lit for lit in c)]
if dpll_solve(new_clauses, assignment):
    return True
del assignment[-pos_lit]
return False

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    instances_tested = 30
    total_length = 0
    total_sum_abs_coefficients = 0

    for _ in range(instances_tested):
        clauses = generate_3sat_instance(n)
        f = [indicator_function(x, n) for x in product([0, 1], repeat=n)]
        F = fourier_coefficients(f)
        sum_abs_coefficients = sum(abs(coeff) for coeff in F)

        assignment = {}
        length = dpll_solve(clauses, assignment)
        if not length:
            return {
                "metric_name": "resolution_proof_length",
                "metric_value": None,
                "instances_tested": instances_tested,
                "conjecture_holds": False,
                "counterexample": "unsatisfiable_instance"
            }

        total_length += length
        total_sum_abs_coefficients += sum_abs_coefficients

    mean_length = total_length / instances_tested
    mean_sum_abs_coefficients = total_sum_abs_coefficients / instances_tested
    conjecture_holds = mean_length >= mean_sum_abs_coefficients

    return {
        "metric_name": "resolution_proof_length",
        "metric_value": mean_length,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    from itertools import product

```

```

seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_length = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(r
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_length} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_length} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Increase timeout duration and re-run tests with diverse seed configurations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	106679
2	preregistration	ollama_remote	qwen3:8b	0	0	23779
3	novelty	ollama_remote	qwen3:8b	0	0	22094
4	novelty	ollama_remote	qwen3:8b	0	0	17204
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12796
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12502

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	judge	ollama_remote	qwen3:8b	0	0	15613

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 210667 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in research/audit/161db2211085.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/161db2211085.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/161db2211085.tar.gz (if generated)